```
Start coding or generate with AI.
```

A. Brief Explanation of the Approach

This notebook will demonstrate two clustering algorithms: Minimum Spanning Tree (MST) based clustering and K-Means clustering, applied to the Iris dataset. The goal is to compare their performance, especially noting MST's ability to handle non-spherical clusters.

MST-based Clustering: This method first constructs a Minimum Spanning Tree from the data points, where edges represent distances between points. To form `k` clusters, the `k-1` longest edges in the MST are removed. The connected components that remain after removing these edges then form the clusters. This approach naturally discovers clusters of arbitrary shapes.

K-Means Clustering: This is a centroid-based algorithm that partitions data into `k` clusters, aiming to minimize the variance within each cluster. It's effective for spherical, well-separated clusters.

Both methods will be evaluated using the Silhouette Score, and their cluster assignments will be exported to CSV files. The features of the Iris dataset will be standardized prior to clustering to ensure fair distance calculations.

B. Import Libraries

```
import pandas as pd
import numpy as np
import networkx as nx
from scipy.spatial.distance import pdist, squareform
from scipy.sparse.csgraph import minimum_spanning_tree, connected_components
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import os
```

C. Load Data

We will load the Iris dataset. Since `iris.zip` is available, we'll unzip it and load `iris.csv`. If it were not present, code for direct download from UCI or manual upload would be needed.

```
# Check if iris.zip exists and unzip it
if os.path.exists('iris.zip'):
    print("Unzipping iris.zip...")
    !unzip -o iris.zip # -o option to overwrite files without prompting
    print("iris.zip unzipped.")
else:
    print("iris.zip not found. Please ensure it's in the current directory or manually upload iris.data.")

# Define the path to the iris data file
# The unzipped file is typically named 'iris.data', not 'iris.csv'
iris_data_path = 'iris.data'

# Load the dataset
try:
    # The Iris dataset typically does not have a header in the raw UCI version.
    # We'll assign standard column names for the features and species.
    feature_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
    df = pd.read_csv(iris_data_path, header=None, names=feature_names + ['species'])
    print(f"Successfully loaded data from {iris_data_path}")
except FileNotFoundError:
    print(f"Error: {iris_data_path} not found. Please make sure the file is available.")
    # As an alternative, if iris.data is not present, one could fetch it directly
    # from the UCI repository or use scikit-learn's built-in iris dataset.
    # Example using scikit-learn (if iris.data is not available and needed to proceed):
    # from sklearn.datasets import load_iris
    # iris = load_iris()
    # df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
    # df['species'] = iris.target_names[iris.target]
    # print("Loaded Iris dataset from scikit-learn.")
    exit() # Exit if data cannot be loaded

# Create a sample ID column starting from 1
df.insert(0, 'SampleId', range(1, 1 + len(df)))

# Use only numeric feature columns for clustering (ignore 'species' column)
feature_cols = feature_names # Already defined above
X = df[feature_cols]
```

```
display(df.head())
```

```
Unzipping iris.zip...
Archive:  iris.zip
  inflating: Index
  inflating: bezdekIris.data
  inflating: iris.data
  inflating: iris.names
iris.zip unzipped.
Successfully loaded data from iris.data
```

	SampleId	sepal_length	sepal_width	petal_length	petal_width	species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

D. Preprocessing: Standardize Features

Standardizing features ensures that all features contribute equally to the distance calculations, preventing features with larger scales from dominating the process.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Features standardized. Shape of scaled data:", X_scaled.shape)
print("First 5 rows of scaled features:")
display(pd.DataFrame(X_scaled, columns=feature_cols).head())
```

```
Features standardized. Shape of scaled data: (150, 4)
First 5 rows of scaled features:
```

	sepal_length	sepal_width	petal_length	petal_width
0	-0.900681	1.032057	-1.341272	-1.312977
1	-1.143017	-0.124958	-1.341272	-1.312977
2	-1.385353	0.337848	-1.398138	-1.312977
3	-1.506521	0.106445	-1.284407	-1.312977
4	-1.021849	1.263460	-1.341272	-1.312977

E. MST Clustering Function

This section implements the MST-based clustering algorithm. It calculates pairwise Euclidean distances, builds a Minimum Spanning Tree, removes the `k-1` longest edges, and then identifies connected components as clusters. We'll use `scipy` for distance and MST computation, and `networkx` to easily find connected components after edge removal.

```
def mst_clustering(X, k):
    """
    Performs MST-based clustering on the given data.

    Args:
        X (np.ndarray): The input data (features).
        k (int): The number of clusters to form.

    Returns:
        np.ndarray: An array of cluster labels for each sample.
    """
    if k < 2:
        raise ValueError("k must be at least 2 for clustering.")

    # 1. Compute pairwise Euclidean distances
    # pdist computes condensed distance matrix, squareform converts to square matrix
    dist_matrix = squareform(pdist(X, metric='euclidean'))

    # 2. Build a complete weighted graph conceptually from the distances
    # 3. Construct the Minimum Spanning Tree using scipy's efficient implementation
    # This returns a sparse matrix representing the MST.
    mst = minimum_spanning_tree(dist_matrix)

    # Extract edges and their weights from the MST (correct way for csr_array)
    mst_edges = []
```

```
    mst_edges = []
    # Iterate through the non-zero elements of the sparse matrix to get edges
    # For a sparse matrix, the non-zero entries correspond to edges in the MST
    rows, cols = mst.nonzero()
    for r, c in zip(rows, cols):
        # Ensure each edge is added only once (undirected graph) and has a valid weight
        if r < c: # Avoid duplicate edges (e.g., (0,1) and (1,0)) and self-loops
            weight = mst[r, c]
            mst_edges.append((r, c, weight))

    # 4. Sort MST edges by weight in descending order
    mst_edges.sort(key=lambda x: x[2], reverse=True)

    # 5. Remove the (k-1) most expensive edges from the MST to form k clusters
    # If k clusters are desired, k-1 edges must be removed.
    # If k is too large (e.g., k > num_samples), we might remove all edges, resulting in single-point clusters.
    num_edges_to_remove = min(k - 1, len(mst_edges))
    edges_to_keep = mst_edges[num_edges_to_remove:]

    # Build a new graph from the remaining edges
    G_clustered = nx.Graph()
    # Add all nodes first, even if some become isolated after edge removal
    G_clustered.add_nodes_from(range(X.shape[0]))
    G_clustered.add_edges_from([(u, v) for u, v, w in edges_to_keep])

    # 6. Assign cluster labels from the connected components
    # Use networkx to find connected components easily
    clusters = list(nx.connected_components(G_clustered))

    labels = np.zeros(X.shape[0], dtype=int)
    for cluster_id, component in enumerate(clusters):
        for node_idx in component:
            labels[node_idx] = cluster_id

    # Handle cases where k is greater than the number of connected components formed
    # The actual number of clusters might be less than k if there aren't enough edges to remove,
    # or if the graph naturally forms fewer components after removing k-1 edges.
    # The silhouette score calculation will correctly use the actual clusters formed.

    print(f"MST clustering completed. Formed {len(clusters)} clusters.")
    return labels

# Define k for clustering (Iris dataset typically has 3 classes)
k_clusters = 3

# Run MST clustering
mst_labels = mst_clustering(X_scaled, k_clusters)
print("MST Cluster Labels (first 10 samples):", mst_labels[:10])
```

```
MST clustering completed. Formed 3 clusters.
MST Cluster Labels (first 10 samples): [0 0 0 0 0 0 0 0 0 0]
```

F. K-Means Clustering

We will run K-Means with the same `k` value to ensure a fair comparison.

```
# Run K-Means clustering
kmeans = KMeans(n_clusters=k_clusters, random_state=42, n_init=10) # n_init for robust centroid initialization
kmeans_labels = kmeans.fit_predict(X_scaled)

print("K-Means Cluster Labels (first 10 samples):", kmeans_labels[:10])
```

```
K-Means Cluster Labels (first 10 samples): [1 1 1 1 1 1 1 1 1 1]
```

G. Evaluation and CSV Export

We will calculate the Silhouette Score for both clustering methods and then export their respective cluster labels to CSV files.

```
# Compute Silhouette Score for MST clustering
# Check if more than 1 cluster is formed and data points > 1
if len(np.unique(mst_labels)) > 1 and len(X_scaled) > 1:
    mst_silhouette = silhouette_score(X_scaled, mst_labels)
else:
    mst_silhouette = -1 # Or np.nan, indicating silhouette is not applicable
    print("Warning: MST clustering resulted in 1 or fewer unique clusters, Silhouette Score not applicable.")

# Compute Silhouette Score for K-Means clustering
if len(np.unique(kmeans_labels)) > 1 and len(X_scaled) > 1:
    kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
```

```
else:
    kmeans_silhouette = -1 # Or np.nan
    print("Warning: K-Means clustering resulted in 1 or fewer unique clusters, Silhouette Score not applicable.")

# Prepare data for export
mst_clusters_df = pd.DataFrame({'SampleId': df['SampleId'], 'ClusterLabel': mst_labels})
kmeans_clusters_df = pd.DataFrame({'SampleId': df['SampleId'], 'ClusterLabel': kmeans_labels})

# Export to CSV
mst_clusters_df.to_csv('mst_clusters.csv', index=False)
kmeans_clusters_df.to_csv('kmeans_clusters.csv', index=False)

print(f"\nSilhouette Score for MST Clustering: {mst_silhouette:.4f}")
print(f"Silhouette Score for K-Means Clustering: {kmeans_silhouette:.4f}")
print("MST cluster labels exported to mst_clusters.csv")
print("K-Means cluster labels exported to kmeans_clusters.csv")

print("\nFirst 5 rows of mst_clusters.csv:")
display(mst_clusters_df.head())
print("\nFirst 5 rows of kmeans_clusters.csv:")
display(kmeans_clusters_df.head())
```

```
Silhouette Score for MST Clustering: 0.5029
Silhouette Score for K-Means Clustering: 0.4590
MST cluster labels exported to mst_clusters.csv
K-Means cluster labels exported to kmeans_clusters.csv

First 5 rows of mst_clusters.csv:
```

	SampleId	ClusterLabel
0	1	0
1	2	0
2	3	0
3	4	0
4	5	0

```
First 5 rows of kmeans_clusters.csv:
```

	SampleId	ClusterLabel
0	1	1
1	2	1
2	3	1
3	4	1
4	5	1

H. Final Comparison Summary

Here's a brief comparison of the two clustering methods based on their Silhouette Scores and general characteristics.

```
print("--- Clustering Comparison Report ---")
print(f"Silhouette Score (MST Clustering): {mst_silhouette:.4f}")
print(f"Silhouette Score (K-Means Clustering): {kmeans_silhouette:.4f}")
print("\nBrief Note:")
if mst_silhouette > kmeans_silhouette:
    print("MST clustering achieved a higher Silhouette Score than K-Means, suggesting better-defined and more separated clu
    print("This might indicate that the natural clusters in Iris, while somewhat spherical, benefit from MST's ability to d
elif kmeans_silhouette > mst_silhouette:
    print("K-Means clustering achieved a higher Silhouette Score than MST, suggesting it found more compact and well-separa
    print("This result is typical for datasets where clusters are indeed spherical or convex, which is often the case with
else:
    print("Both MST and K-Means clustering achieved similar Silhouette Scores.")
    print("This could imply that the clusters are relatively simple and well-separated, making both methods perform compara
```

```
--- Clustering Comparison Report ---
Silhouette Score (MST Clustering): 0.5029
Silhouette Score (K-Means Clustering): 0.4590

Brief Note:
MST clustering achieved a higher Silhouette Score than K-Means, suggesting better-defined and more separated clusters accord
This might indicate that the natural clusters in Iris, while somewhat spherical, benefit from MST's ability to delineate bou
```

I. 8-10 Line Explanation of MST vs K-Means for Non-Spherical Clusters

Minimum Spanning Tree (MST) based clustering can better capture non-spherical clusters than K-Means because its mechanism fundamentally differs. K-Means relies on centroids and assumes clusters are convex and isotropic (spherical-like), assigning points to the nearest centroid. This struggles with elongated, crescent-shaped, or intertwined clusters. In contrast, MST clustering connects all data points with minimal total edge weight, effectively identifying the 'skeleton' of the data distribution. By strategically removing the longest edges, it cuts across sparse regions, naturally separating dense components regardless of their geometric shape. This ability to delineate clusters based on connectivity rather than geometric proximity to a central point allows MST to discover arbitrary-shaped clusters that K-Means would fail to correctly segment, often splitting single non-spherical clusters into multiple 'spherical' parts.